There’s no requirement that the userinfo component have the format username:password, so you can just include a token directly instead:

```
https://gF4PI4M2f8qJOjfGV5LfXJp0CHM@localhost:4567/spaces
```

If you are using this format with JavaScript clients, then the token will be available as the `username` field of the `Location` object (which is implemented by `window.location`, `document.location`, and anchor tag elements). Alternatively, you can prefix the token with a colon to leave the `username` field blank and instead make it available as the `password` field, which more closely reflects how it is being used: no user is involved, but the token is a security credential a bit like a password. Listing 9.8 shows how to adapt the `CapabilityController` to return URIs in this format by making use of the URI constructor that takes the scheme, userinfo, hostname, port, path, query, and fragment components as separate arguments. You can also change the `lookupPermissions` method to check for the token in the Authorization header rather than in a query parameter to better protect it against accidental leaks. Open CapabilityController.java and update the methods to match the listing.

Listing 9.8 Using the userinfo URI component

```
public URI createUri(Request request, String path, String
perms) {
var token = new Token(Instant.MAX, null);
token.attributes.put("path", path);
token.attributes.put("perms", perms);
```